Experiment 18: RNN, LSTM and GRU Hyperparameter Experimentation

1. Aim

To implement and compare SimpleRNN, LSTM, and GRU models for binary sentiment classification of movie reviews using the IMDB dataset. To tokenize and pad text reviews, apply an embedding layer, and automatically optimize hyperparameters such as hidden units, number of layers, dropout rate, learning rate, and embedding dimensions using KerasTuner. To identify the best configuration for each recurrent architecture and compare their accuracy, trainable parameter count, and training time using TensorFlow, Keras, NumPy, KerasTuner, and Matplotlib.

2. Mathematical Formulas

1. Word Embedding

$$E \in \mathbb{R}^{V \times d}$$

Where V is vocabulary size and d is embedding dimension.

2. SimpleRNN Hidden State

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

3. LSTM Forget Gate

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

4. LSTM Input Gate

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$$

5. LSTM Cell State

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

6. LSTM Output Gate

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

7. GRU Update Gate

$$z_t = \sigma(W_z [h_{t-1}, x_t] + b_z)$$

8. GRU Reset Gate

$$r_t = \sigma(W_r [h_{t-1}, x_t] + b_r)$$

9. GRU Hidden State

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

CODE

```
# Experiment 18: RNN, LSTM and GRU Hyperparameter Experimentation

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
import keras_tuner as kt
import time

from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Embedding, SimpleRNN, LSTM, GRU,
    Dense, Dropout
)

np.random.seed(42)
tf.random.set_seed(42)

# ========================================
# 1. LOAD IMDB DATASET
# ========================================

vocab_size = 10000
max_length = 200

(X_train_full, y_train_full), (X_test, y_test) = (
    imdb.load_data(num_words=vocab_size)
)

print("Training Reviews:", len(X_train_full))
print("Testing Reviews:", len(X_test))

# ========================================
# 2. PAD THE SEQUENCES
# ========================================

X_train_full = pad_sequences(
    X_train_full,
    maxlen=max_length,
    padding="post",
    truncating="post"
)

X_test = pad_sequences(
    X_test,
    maxlen=max_length,
    padding="post",
    truncating="post"
)

# Create training and validation sets
X_train = X_train_full[:20000]
y_train = y_train_full[:20000]

X_val = X_train_full[20000:]
```

```
y_val = y_train_full[20000:]

print("\nTraining Shape:", X_train.shape)
print("Validation Shape:", X_val.shape)
print("Testing Shape:", X_test.shape)

# ========================================
# 3. DISPLAY SAMPLE REVIEWS
# ========================================

word_index = imdb.get_word_index()

reverse_word_index = {
    value + 3: key
    for key, value in word_index.items()
}

reverse_word_index[0] = "<PAD>"
reverse_word_index[1] = "<START>"
reverse_word_index[2] = "<UNK>"

def decode_review(sequence):
    return " ".join(
        reverse_word_index.get(i, "?")
        for i in sequence if i != 0
    )

print("\nSample Review:")
print(decode_review(X_train[0])[:500])

print("\nSentiment:",
      "Positive" if y_train[0] == 1 else "Negative")

# ========================================
# 4. DEFINE MODEL BUILDER
# ========================================

def model_builder(hp, cell_type):

    model = Sequential()

    embedding_dim = hp.Choice(
        "embedding_dim",
        values=[32, 64]
    )

    model.add(
        Embedding(
            input_dim=vocab_size,
            output_dim=embedding_dim,
            input_length=max_length
        )
    )

    units = hp.Choice(
        "units",
        values=[32, 64]
    )
```

```
    num_layers = hp.Choice(
        "num_layers",
        values=[1, 2]
    )

    dropout_rate = hp.Choice(
        "dropout",
        values=[0.0, 0.2, 0.3]
    )

    # Select recurrent architecture
    if cell_type == "RNN":
        layer_class = SimpleRNN

    elif cell_type == "LSTM":
        layer_class = LSTM

    else:
        layer_class = GRU

    # Add recurrent layers
    for i in range(num_layers):

        return_sequences = (i < num_layers - 1)

        model.add(
            layer_class(
                units=units,
                activation="tanh",
                return_sequences=return_sequences
            )
        )

        if dropout_rate > 0:
            model.add(
                Dropout(dropout_rate)
            )

    # Binary classification output
    model.add(
        Dense(1, activation="sigmoid")
    )

    learning_rate = hp.Choice(
        "learning_rate",
        values=[0.001, 0.0005]
    )

    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=learning_rate
        ),
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    return model
```

```python
# ========================================
# 5. HYPERPARAMETER TUNING
# ========================================

cell_types = ["RNN", "LSTM", "GRU"]

best_models = {}
best_hps = {}
tuning_results = {}

for cell_type in cell_types:

    print("\n================================")
    print("TUNING:", cell_type)
    print("================================")

    tuner = kt.RandomSearch(
        hypermodel=lambda hp: model_builder(
            hp, cell_type
        ),
        objective="val_accuracy",
        max_trials=5,
        executions_per_trial=1,
        directory="keras_tuner_results",
        project_name=cell_type,
        overwrite=True,
        seed=42
    )

    tuner.search(
        X_train,
        y_train,
        validation_data=(X_val, y_val),
        epochs=3,
        batch_size=64,
        verbose=1
    )

    # Retrieve best configuration
    best_hp = tuner.get_best_hyperparameters(
        num_trials=1
    )[0]

    best_model = tuner.get_best_models(
        num_models=1
    )[0]

    best_hps[cell_type] = best_hp
    best_models[cell_type] = best_model

    tuning_results[cell_type] = {
        "Units": best_hp.get("units"),
        "Layers": best_hp.get("num_layers"),
        "Dropout": best_hp.get("dropout"),
        "Embedding": best_hp.get("embedding_dim"),
        "Learning Rate": best_hp.get("learning_rate")
    }
```

```
    print("\nBest Hyperparameters:")
    for key, value in tuning_results[cell_type].items():
        print(key, ":", value)

# ========================================
# 6. RETRAIN BEST MODELS
# ========================================

histories = {}
training_times = {}
model_parameters = {}

for cell_type in cell_types:

    print("\nTraining Best", cell_type, "Model")

    model = best_models[cell_type]

    start_time = time.perf_counter()

    history = model.fit(
        X_train,
        y_train,
        validation_data=(X_val, y_val),
        epochs=5,
        batch_size=64,
        verbose=1
    )

    elapsed = time.perf_counter() - start_time

    histories[cell_type] = history
    training_times[cell_type] = elapsed
    model_parameters[cell_type] = model.count_params()

    print("Training Time:", round(elapsed, 2), "seconds")
    print("Parameters:", model.count_params())

# ========================================
# 7. EVALUATE ALL MODELS
# ========================================

comparison = []

for cell_type in cell_types:

    model = best_models[cell_type]

    test_loss, test_accuracy = model.evaluate(
        X_test,
        y_test,
        verbose=0
    )

    comparison.append({
        "Model": cell_type,
        "Test Accuracy (%)": test_accuracy * 100,
```

```
        "Test Loss": test_loss,
        "Parameters": model_parameters[cell_type],
        "Training Time (seconds)": training_times[cell_type]
    })

    print("\n", cell_type, "RESULT")
    print("Test Accuracy:", round(test_accuracy * 100, 2), "%")
    print("Test Loss:", round(test_loss, 4))

# ========================================
# 8. DISPLAY HYPERPARAMETER TABLE
# ========================================

hp_table = pd.DataFrame(tuning_results).T

print("\nBEST HYPERPARAMETERS")
print(hp_table.to_string())

# ========================================
# 9. DISPLAY MODEL COMPARISON TABLE
# ========================================

comparison_df = pd.DataFrame(comparison)

print("\nMODEL PERFORMANCE COMPARISON")
print(comparison_df.round(3).to_string(index=False))

# ========================================
# 10. PLOT TRAINING AND VALIDATION ACCURACY
# ========================================

plt.figure(figsize=(10, 5))

for cell_type in cell_types:

    history = histories[cell_type]

    plt.plot(
        history.history["accuracy"],
        label=cell_type + " Training"
    )

    plt.plot(
        history.history["val_accuracy"],
        linestyle="--",
        label=cell_type + " Validation"
    )

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("RNN, LSTM and GRU Accuracy Comparison")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# ========================================
# 11. PLOT TRAINING AND VALIDATION LOSS
# ========================================
```

```
plt.figure(figsize=(10, 5))

for cell_type in cell_types:

    history = histories[cell_type]

    plt.plot(
        history.history["loss"],
        label=cell_type + " Training"
    )

    plt.plot(
        history.history["val_loss"],
        linestyle="--",
        label=cell_type + " Validation"
    )

plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy Loss")
plt.title("Training and Validation Loss Comparison")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# ========================================
# 12. PLOT TEST ACCURACY
# ========================================

plt.figure(figsize=(8, 5))

plt.bar(
    comparison_df["Model"],
    comparison_df["Test Accuracy (%)"]
)

plt.ylabel("Test Accuracy (%)")
plt.title("Test Accuracy Comparison")
plt.grid(axis="y", alpha=0.3)
plt.show()

# ========================================
# 13. PLOT PARAMETER COUNT
# ========================================

plt.figure(figsize=(8, 5))

plt.bar(
    comparison_df["Model"],
    comparison_df["Parameters"]
)

plt.ylabel("Number of Parameters")
plt.title("Model Parameter Comparison")
plt.grid(axis="y", alpha=0.3)
plt.show()

# ========================================
```

```
# 14. PLOT TRAINING TIME
# ========================================

plt.figure(figsize=(8, 5))

plt.bar(
    comparison_df["Model"],
    comparison_df["Training Time (seconds)"]
)

plt.ylabel("Training Time (seconds)")
plt.title("Training Time Comparison")
plt.grid(axis="y", alpha=0.3)
plt.show()

# ========================================
# 15. IDENTIFY BEST MODEL
# ========================================

best_model_row = comparison_df.loc[
    comparison_df["Test Accuracy (%)"].idxmax()
]

print("\nBEST MODEL:")
print(best_model_row.to_string())

print("\nExperiment Completed Successfully!")
```

OUTPUT

```
Trial 5 Complete [00h 03m 33s]
val_accuracy: 0.8321999907493591

Best val_accuracy So Far: 0.8321999907493591
Total elapsed time: 00h 19m 40s

Best Hyperparameters:
Units : 32
Layers : 1
Dropout : 0.2
Embedding : 64
Learning Rate : 0.001

Training Best RNN Model
Epoch 1/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 23s 68ms/step - accuracy: 0.5766 - loss: 0.6716 - val_accuracy: 0.5260 - val_loss: 0.6934
Epoch 2/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 51s 99ms/step - accuracy: 0.6327 - loss: 0.6233 - val_accuracy: 0.5054 - val_loss: 0.7090
Epoch 3/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 31s 65ms/step - accuracy: 0.6352 - loss: 0.5938 - val_accuracy: 0.5006 - val_loss: 0.7446
Epoch 4/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 23s 75ms/step - accuracy: 0.6848 - loss: 0.5110 - val_accuracy: 0.5118 - val_loss: 0.8338
Epoch 5/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 20s 64ms/step - accuracy: 0.7241 - loss: 0.4447 - val_accuracy: 0.5018 - val_loss: 0.9325
Training Time: 147.91 seconds
Parameters: 326273

Training Best LSTM Model
Epoch 1/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 100s 307ms/step - accuracy: 0.8113 - loss: 0.4476 - val_accuracy: 0.7986 - val_loss: 0.4812
Epoch 2/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 146s 320ms/step - accuracy: 0.8416 - loss: 0.4116 - val_accuracy: 0.7502 - val_loss: 0.5995
Epoch 3/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 98s 313ms/step - accuracy: 0.8544 - loss: 0.3760 - val_accuracy: 0.8384 - val_loss: 0.4013
Epoch 4/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 143s 316ms/step - accuracy: 0.7898 - loss: 0.4675 - val_accuracy: 0.6462 - val_loss: 0.6249
Epoch 5/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 94s 301ms/step - accuracy: 0.7320 - loss: 0.5317 - val_accuracy: 0.7536 - val_loss: 0.5869
Training Time: 628.74 seconds
Parameters: 706113

Training Best GRU Model
Epoch 1/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 47s 144ms/step - accuracy: 0.8908 - loss: 0.2815 - val_accuracy: 0.8634 - val_loss: 0.3418
Epoch 2/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 45s 145ms/step - accuracy: 0.9304 - loss: 0.1950 - val_accuracy: 0.8708 - val_loss: 0.3628
Epoch 3/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 46s 147ms/step - accuracy: 0.9518 - loss: 0.1442 - val_accuracy: 0.8428 - val_loss: 0.4292
Epoch 4/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 81s 145ms/step - accuracy: 0.9622 - loss: 0.1141 - val_accuracy: 0.8472 - val_loss: 0.4067
Epoch 5/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 45s 142ms/step - accuracy: 0.9732 - loss: 0.0850 - val_accuracy: 0.8512 - val_loss: 0.4637
```

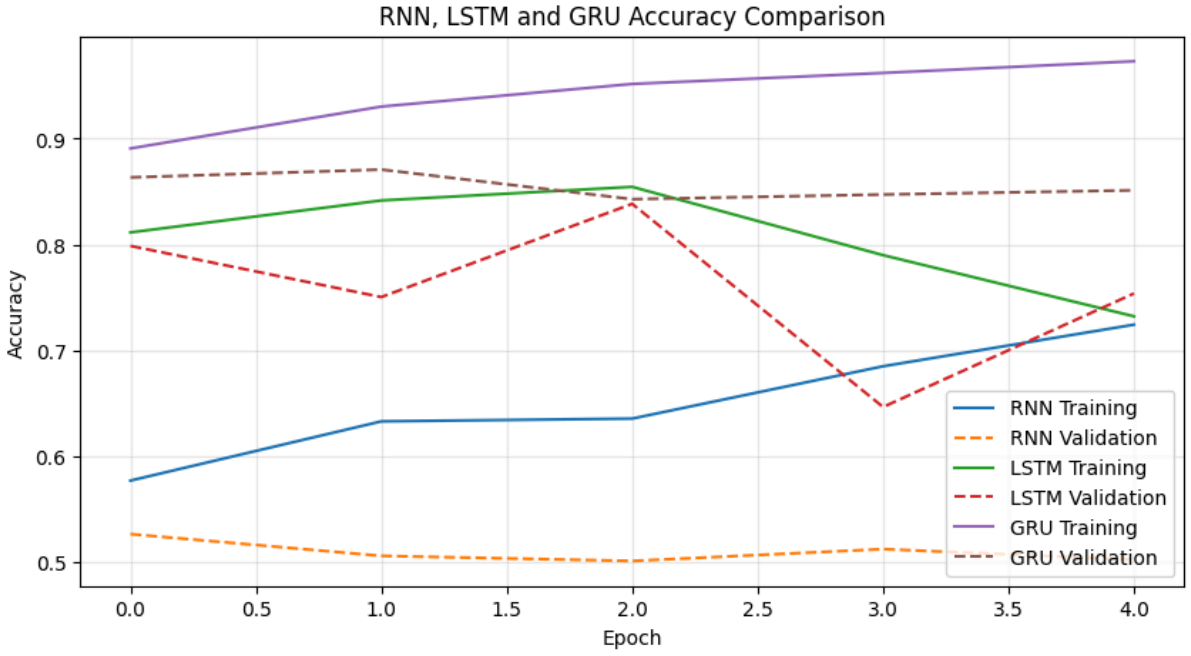
```
 RNN RESULT
Test Accuracy: 49.8 %
Test Loss: 0.923

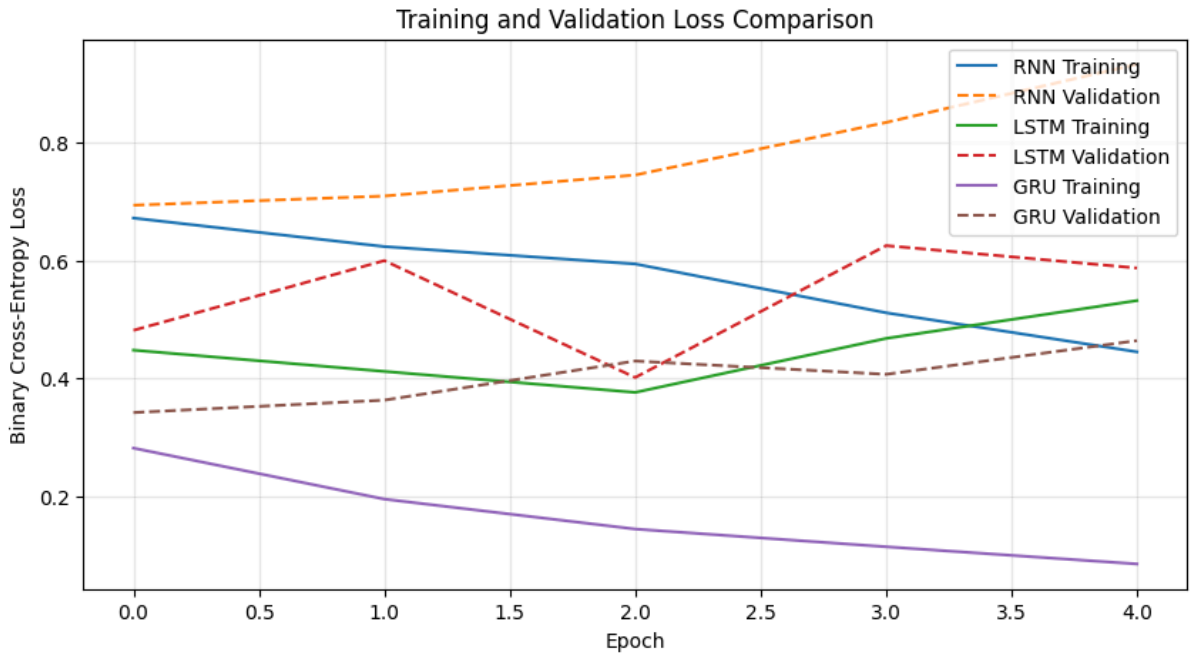
 LSTM RESULT
Test Accuracy: 73.39 %
Test Loss: 0.6035

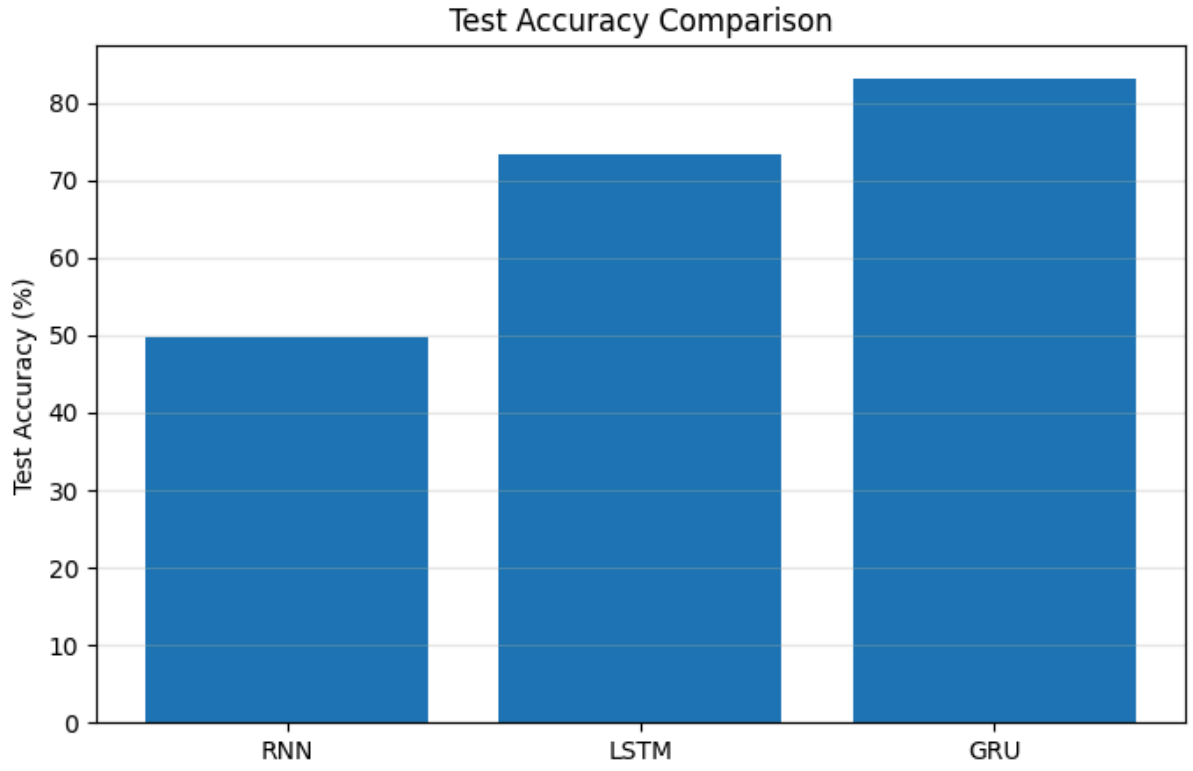
 GRU RESULT
Test Accuracy: 83.16 %
Test Loss: 0.5341

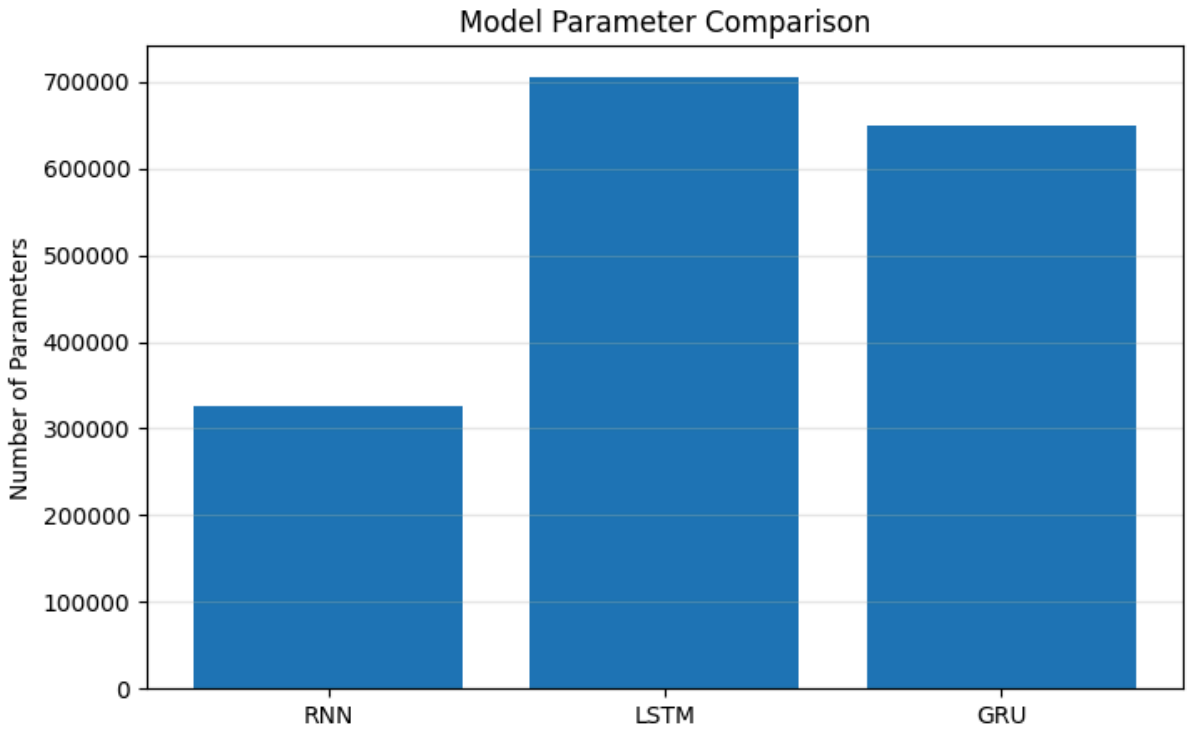
BEST HYPERPARAMETERS
      Units  Layers  Dropout  Embedding  Learning Rate
RNN    64.0     1.0      0.2       32.0         0.0010
LSTM   64.0     2.0      0.0       64.0         0.0005
GRU    32.0     1.0      0.2       64.0         0.0010

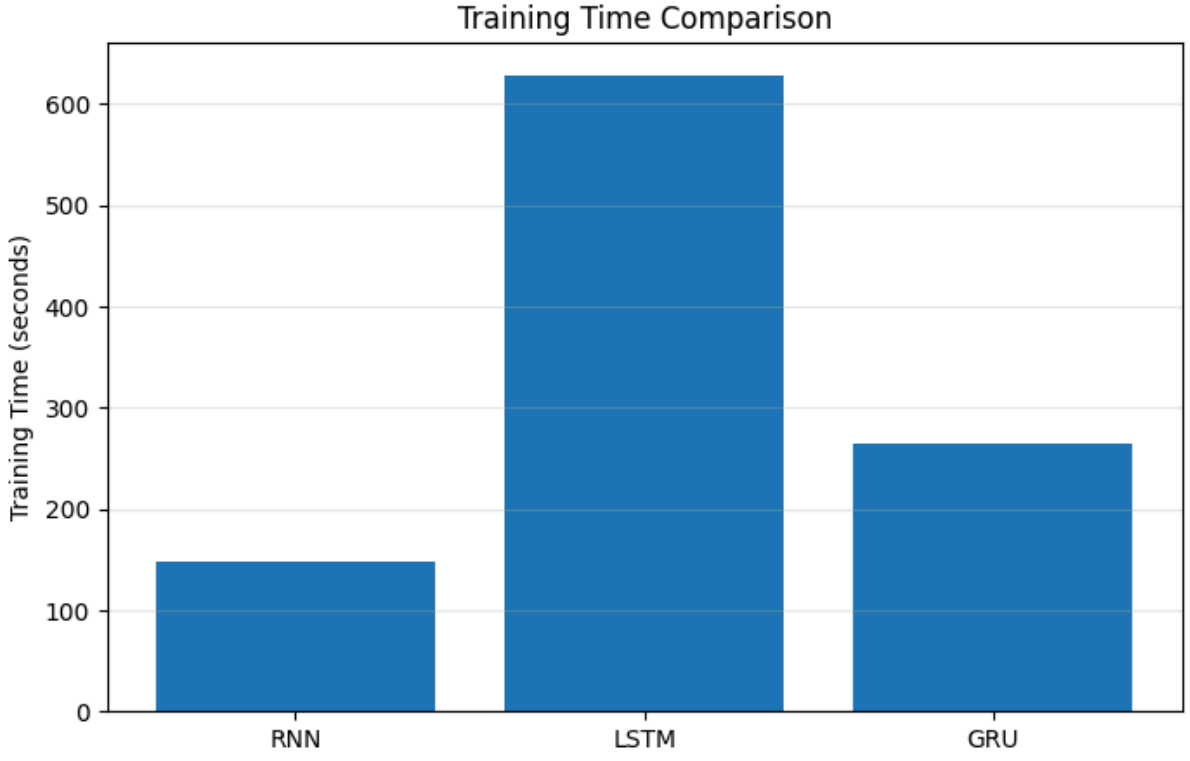
MODEL PERFORMANCE COMPARISON
Model  Test Accuracy (%)  Test Loss  Parameters  Training Time (seconds)
  RNN             49.800      0.923      326273                 147.913
 LSTM             73.388      0.603      706113                 628.738
  GRU             83.156      0.534      649441                 264.746
```




Training and Validation Loss Comparison
RNN Training
RNN Validation
LSTM Training
LSTM Validation
GRU Training
GRU Validation
Binary Cross-Entropy Loss
0.2
0.4
0.6
0.8
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epoch


Test Accuracy Comparison
Test Accuracy (%)
0
10
20
30
40
50
60
70
80
RNN
LSTM
GRU


Model Parameter Comparison
Number of Parameters
0
100000
200000
300000
400000
500000
600000
700000
RNN
LSTM
GRU


Training Time Comparison
Training Time (seconds)
0
100
200
300
400
500
600
RNN
LSTM
GRU

```
BEST MODEL:
Model                              GRU
Test Accuracy (%)           83.156002
Test Loss                     0.53412
Parameters                     649441
Training Time (seconds)    264.746005

Experiment Completed Successfully!
```

5. RESULT

SimpleRNN, LSTM, and GRU models were implemented using TensorFlow and Keras to classify IMDB movie reviews into positive and negative categories. Reviews were tokenized, padded, and converted into embeddings. KerasTuner was used to automatically optimize the embedding dimension, hidden units, number of layers, dropout rate, and learning rate for each architecture. The three models were evaluated using test accuracy, parameter count, and training time. Accuracy and training comparison graphs were generated using Matplotlib. Thus, the experiment demonstrates automated hyperparameter optimization and the performance differences between RNN, LSTM, and GRU architectures for sentiment classification.